



UVA 10600

ACM Contest and Blackout

João Isaías - 2310283 | Luiz Carlos - 2410410 | Ricardo André - 2417200

CONTEXTO

CONTEXTO

Durante a preparação de um concurso nacional da ACM, a prefeitura decidiu garantir uma fonte confiável de energia para todas as escolas da cidade, evitando possíveis apagões.

Para isso, a usina geradora “Future” deve estar conectada a pelo menos uma escola, e as demais escolas podem receber energia diretamente da usina ou indiretamente através de outras escolas já conectadas.

OBJETIVO

Cada possível conexão possui um **custo associado**, e o objetivo é encontrar os dois planos de conexão de **menor custo possível**.



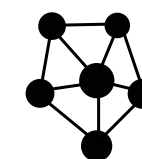
Escolas

Representam os vértices do grafo que precisam receber energia de forma confiável.



Usinas

Representam a fonte geradora de energia (“Future”), responsável por iniciar a distribuição elétrica para as escolas.



Conexões

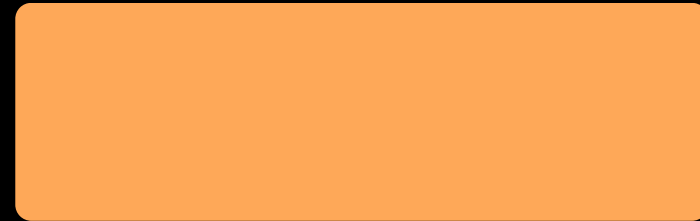
Representam as arestas do grafo, ligando escolas e usinas, onde cada conexão possui um custo associado.

Entrada

Saída

Sample Input

```
2
5 8
1 3 75
3 4 51
2 4 19
3 2 95
2 5 42
5 4 31
1 2 9
3 5 66
```



Entrada

Sample Input

2 ◦

5 8

1 3 75

3 4 51

2 4 19

3 2 95

2 5 42

5 4 31

1 2 9

3 5 66

A entrada começa com um
inteiro T, representando o número
de **casos de teste**.

Entrada

Sample Input

2

5 8 0

1 3 75

3 4 51

2 4 19

3 2 95

2 5 42

5 4 31

1 2 9

3 5 66

A entrada começa com um
inteiro T , representando o número
de casos de teste.

Uma linha contendo dois inteiros:
 $N \rightarrow$ número de escolas da cidade;
 $M \rightarrow$ número de conexões possíveis entre elas.

Entrada

Sample Input

2

5 8

1 3 75

3 4 51

2 4 19

3 2 95

2 5 42

5 4 31

1 2 9

3 5 66

A entrada começa com um inteiro T , representando o número de casos de teste.

Uma linha contendo dois inteiros:

$N \rightarrow$ número de escolas da cidade;

$M \rightarrow$ número de conexões possíveis entre elas.

Em seguida, existem M linhas contendo três inteiros:
 A_i, B_i, C_i

Entrada

Sample Input

```
2
5 8
1 3 75
3 4 51
2 4 19
3 2 95
2 5 42
5 4 31
1 2 9
3 5 66
```

A entrada começa com um inteiro T , representando o número de casos de teste.

Uma linha contendo dois inteiros:
 $N \rightarrow$ número de escolas da cidade;
 $M \rightarrow$ número de conexões possíveis entre elas.

Em seguida, existem M linhas contendo três inteiros:
 A_i, B_i, C_i

A_i e $B_i \rightarrow$ duas escolas conectadas;
 $C_i \rightarrow$ o custo da conexão entre essas escolas.

Entrada

Entrada

Saída

Saída

Sample Output

110 121

37 37

Para cada caso de teste, uma única linha contendo dois números separados por espaço:

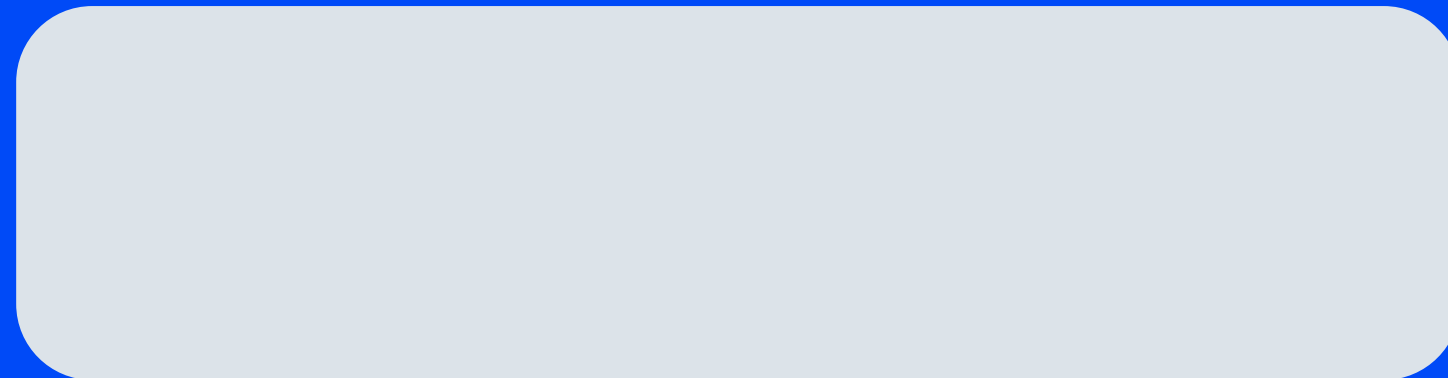
S1 → custo do plano de conexão mais barato;

S2 → custo do segundo plano de conexão mais barato.

S1 = S2 somente quando existirem duas soluções mínimas diferentes com o mesmo custo;
caso contrário, **S1 < S2**

algoritmo de Kruskal

algoritmo de Kruskal

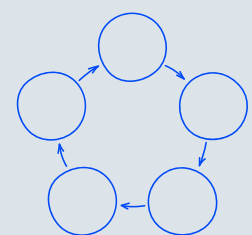


algoritmo de Kruskal



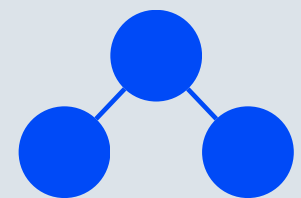
ordena todas as arestas pelo menor peso adicionando as conexões gradualmente

algoritmo de Kruskal



evita ciclos através da
estrutura Union-Find

algoritmo de Kruskal



constrói a MST de
maneira gulosa

algoritmo de Kruskal

Resolução da MST (S1)

algoritmo de Kruskal

Resolução da MST (S1)

algoritmo de Kruskal

Resolução da MST (S1)

algoritmo de Kruskal

1

Ordenamos as arestas pelo menor custo.

Resolução da MST (S1)

algoritmo de Kruskal

1 Ordenamos as arestas pelo menor custo.

2 Percorremos as conexões em ordem crescente.

Resolução da MST (S1)

algoritmo de Kruskal

- 1 Ordenamos as arestas pelo menor custo.
- 2 Percorremos as conexões em ordem crescente.
- 3 Utilizamos Union-Find para evitar ciclos.

Resolução da MST (S1)

algoritmo de Kruskal

- 1 Ordenamos as arestas pelo menor custo.
- 2 Percorremos as conexões em ordem crescente.
- 3 Utilizamos Union-Find para evitar ciclos.
- 4 Continuamos até conectar todos os vértices.

Resolução da MST (S1)

algoritmo de Kruskal

- 1 Ordenamos as arestas pelo menor custo.
- 2 Percorremos as conexões em ordem crescente.
- 3 Utilizamos Union-Find para evitar ciclos.
- 4 Continuamos até conectar todos os vértices.

Obtemos a Árvore Geradora Mínima (MST)
Custo mínimo total = S1

Resolução da MST (S2)

algoritmo de Kruskal

- 1 Ordenamos as arestas pelo menor custo.
- 2 Percorremos as conexões em ordem crescente.
- 3 Utilizamos Union-Find para evitar ciclos.
- 4 Continuamos até conectar todos os vértices.

Obtemos a Árvore Geradora Mínima (MST)
Custo mínimo total = S1

Resolução da MST (S2)

algoritmo de Kruskal

- 1 Calculamos a MST original.
- 2 Percorremos as conexões em ordem crescente.
- 3 Utilizamos Union-Find para evitar ciclos.
- 4 Continuamos até conectar todos os vértices.

Obtemos a Árvore Geradora Mínima (MST)
Custo mínimo total = S1

Resolução da MST (S2)

algoritmo de Kruskal

- 1 Calculamos a MST original.
- 2 Removemos uma aresta da MST por vez.
- 3 Utilizamos Union-Find para evitar ciclos.
- 4 Continuamos até conectar todos os vértices.

Obtemos a Árvore Geradora Mínima (MST)
Custo mínimo total = S1

Resolução da MST (S2)

algoritmo de Kruskal

1 Calculamos a MST original.

2 Removemos uma aresta da MST por vez.

3 Executamos o Kruskal novamente.

4 Continuamos até conectar todos os vértices.

Obtemos a Árvore Geradora Mínima (MST)
Custo mínimo total = S1

Resolução da MST (S2)

algoritmo de Kruskal

- 1 Calculamos a MST original.
- 2 Removemos uma aresta da MST por vez.
- 3 Executamos o Kruskal novamente.
- 4 Guardamos o menor custo válido encontrado.



Resolução da MST (S2)

algoritmo de Kruskal

- 1 Calculamos a MST original.
- 2 Removemos uma aresta da MST por vez.
- 3 Executamos o Kruskal novamente.
- 4 Guardamos o menor custo válido encontrado.

Menor alternativa válida = S2

Complexidade e Resultado

Complexidade

Ordenação das arestas

$$O(M \log M)$$

Reexecução do Kruskal para a
Segunda MST

$$O(N \cdot M \cdot \alpha(N))$$

Complexidade

Ordenação das arestas

$$O(M \log M)$$

Reexecução do Kruskal para a
Segunda MST

$$O(N \cdot M \cdot \alpha(N))$$

Complexidade

Ordenação das arestas

$$O(M \log M)$$

Reexecução do Kruskal para a
Segunda MST

$$O(N \cdot M \cdot \alpha(N))$$

Complexidade

Ordenação das arestas

$$O(M \log M)$$

Reexecução do Kruskal para a
Segunda MST

$$O(N \cdot M \cdot \alpha(N))$$

$\alpha(N) \rightarrow$ Union-Find

Resultado

Minhas contribuições

#	Problema	Veredicto	Linguagem	Tempo de execução	Data de envio
31149424	10600 Concurso ACM e Blackout	Aceito	PYTH3	0,280	26/05/2026 04:30:05
31149423	10600 Concurso ACM e Blackout	Aceito	PYTH3	0,320	26/05/2026 04:28:56
<< Início < Anterior 1 Próximo > Fim >>					

Mostrar # Resultados 1 - 2 de 2



Obrigado!